海邻到家收银系统 - 社区营销模块详细设计方案

文档版本：v1.0
创建日期：2026-03-06
适用系统：海邻到家智能收银系统

目录

- 一、社区营销模块概述
- 二、业务需求分析
- 三、系统架构设计
- 四、数据库设计
- 五、核心功能实现
- 六、规则引擎设计
- 七、营销活动管理
- 八、数据统计分析
- 九、前端界面设计
- 十、接口设计
- 十一、实施计划

一、社区营销模块概述

1.1 模块定位

海邻到家社区营销模块是专门针对社区便利店场景设计的营销管理系统，通过会员管理、积分体系、优惠券、社区活动等手段，实现社区用户的精细化运营，提升会员粘性、增加复购率、优化社区消费体验。

1.2 核心价值

价值维度	具体体现
提升会员粘性	会员等级体系、积分累计、专属权益
增加复购率	优惠券发放、社区活动、促销推送

价值维度	具体体现
优化消费体验	NFC会员卡、自动折扣、积分自动核销
数据驱动决策	社区消费分析、会员消费习惯分析
社区化运营	社区团购、社区团长、邻里分享

1.3 模块边界

```
社区营销模块边界:
├── 会员管理
│   ├── 会员注册与认证
│   ├── 会员等级管理
│   ├── 会员信息维护
│   └── 会员权益配置
├── 积分管理
│   ├── 积分获取规则
│   ├── 积分使用规则
│   ├── 积分有效期管理
│   └── 积分兑换商城
├── 优惠券管理
│   ├── 优惠券创建与发放
│   ├── 优惠券核销
│   ├── 优惠券统计分析
│   └── 优惠券有效期管理
├── 社区活动管理
│   ├── 活动创建与发布
│   ├── 活动参与与核销
│   ├── 活动统计分析
│   └── 活动模板管理
└── 社区数据分析
    ├── 社区消费分析
    ├── 会员消费习惯分析
    ├── 活动效果分析
    └── 营销ROI分析
```

二、业务需求分析

2.1 社区营销场景

2.1.1 新人场景

```
场景描述：社区新用户首次到店或注册

业务流程：
1. 新用户到店
2. 收银员引导注册会员
3. 注册成功后发放新人礼包
4. 新人礼包包含：新人优惠券、新会员积分、首单优惠

需求点：
- 快速注册流程（手机号/扫码）
- 新人礼包自动发放
- 新人专享商品推荐
- 新人引导提示
```

2.1.2 日常消费场景

```
场景描述：社区会员日常购物消费

业务流程：
1. 会员到店购物
2. NFC刷卡或手机号识别会员
3. 系统自动应用会员折扣
4. 消费积分自动累计
5. 可用优惠券自动提醒
6. 支付完成

需求点：
- 会员识别（NFC/手机号）
- 自动折扣应用
- 积分累计
- 优惠券提醒
- 消费记录同步
```

2.1.3 节日促销场景

```
场景描述：节假日社区促销活动
```

```
业务流程：
1. 创建节日促销活动
2. 配置活动规则（时间/商品/优惠）
3. 活动推广（小程序推送/门店海报）
4. 会员参与活动
5. 活动数据统计分析

需求点：
- 活动快速创建
- 活动规则灵活配置
- 多渠道推广
- 实时数据监控
```

2.1.4 社区团购场景

```
场景描述：社区团长组织团购活动

业务流程：
1. 社区团长创建团购活动
2. 设置团购商品和团购价
3. 社区成员报名参与
4. 团购订单统一配送
5. 团长获得佣金

需求点：
- 团长管理
- 团购活动配置
- 团购订单处理
- 佣金结算
```

2.2 功能需求清单

2.2.1 会员管理

功能编号	功能名称	功能描述	优先级
M001	会员注册	手机号/扫码注册	P0
M002	会员信息维护	姓名/性别/生日/地址	P0
M003	会员等级管理	等级规则/权益配置	P0

功能编号	功能名称	功能描述	优先级
M004	会员权益应用	自动折扣/专属商品	P0
M005	会员卡管理	NFC卡/电子卡/实体卡	P1
M006	会员生日关怀	生日礼品/生日双倍积分	P2
M007	会员推荐奖励	推荐新会员奖励	P2

2.2.2 积分管理

功能编号	功能名称	功能描述	优先级
P001	积分获取	消费积分/签到积分/活动积分	P0
P002	积分使用	抵扣/兑换/抽奖	P0
P003	积分规则	灵活配置获取/使用规则	P0
P004	积分有效期	有效期管理/过期提醒	P1
P005	积分兑换商城	商品兑换/抽奖/秒杀	P2
P006	积分统计	积分趋势/排名	P1

2.2.3 优惠券管理

功能编号	功能名称	功能描述	优先级
C001	优惠券创建	满减券/折扣券/免运费券/商品券	P0
C002	优惠券发放	主动发放/活动发放/领券中心	P0
C003	优惠券核销	自动核销/手动核销	P0
C004	优惠券限制	使用条件/有效期/数量限制	P0
C005	优惠券统计		P1

功能编号	功能名称	功能描述	优先级
		发放统计/使用统计/核销统计	
C006	优惠券分享	好友助力/分享得券	P2

2.2.4 社区活动管理

功能编号	功能名称	功能描述	优先级
A001	活动创建	节日促销/会员日/新人专享	P0
A002	活动规则配置	时间/商品/优惠/限制	P0
A003	活动推广	推送/海报/短信	P1
A004	活动参与	线上报名/线下核销	P0
A005	活动统计	参与人数/转化率/ROI	P1
A006	活动模板	常用活动模板	P2

2.2.5 社区团购

功能编号	功能名称	功能描述	优先级
G001	团长管理	团长招募/团长认证/团长考核	P1
G002	团购活动	创建团购/设置团购价/团购时间	P1
G003	团购订单	团购下单/订单汇总/统一配送	P1
G004	佣金结算	佣金计算/佣金提现/佣金统计	P1
G005	团购推广	团购海报/团购分享	P2

2.2.6 数据分析

功能编号	功能名称	功能描述	优先级
D001	会员分析	会员增长/活跃度/生命周期价值	P1
D002	消费分析	消费频次/客单价/复购率	P1
D003	活动分析	活动参与/活动转化/活动ROI	P1
D004	社区分析	社区消费力/社区偏好/社区对比	P2
D005	商品分析	热销商品/滞销商品/关联商品	P1

三、系统架构设计

3.1 模块架构

```
┌──────────────────────────────────────────────────────────────────┐
│                          社区营销模块架构                          │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│   ┌──────────────────────────────────────────────────────────┐   │
│   │                    表现层 (Presentation)                  │   │
│   ├──────────────────────────────────────────────────────────┤   │
│   │  Web管理端  │  小程序端  │  Android收银端  │  APP端  │   │
│   └──────────────────────────────────────────────────────────┘   │
│                                  │                               │
│   ┌──────────────────────────────┬──────────────────────────┐   │
│   │        应用层 (Application)  │   规则引擎(Rule Engine)  │   │
│   ├──────────────────────────────┼──────────────────────────┤   │
│   │  会员服务   │  积分服务      │  营销规则引擎            │   │
│   │  优惠券服务 │  活动服务      │  积分规则引擎            │   │
│   │  团购服务   │  统计服务      │  优惠券规则引擎          │   │
│   └──────────────────────────────┴──────────────────────────┘   │
│                                  │                               │
│   ┌──────────────────────────────────────────────────────────┐   │
│   │                    领域层 (Domain)                        │   │
```

```
│   ├──────────────────────────────────────────────────────────┤   │
│   │  会员领域  │  积分领域  │  优惠券领域  │  活动领域    │   │
│   │  团购领域  │  规则领域  │  统计领域                   │   │
│   └──────────────────────────────────────────────────────────┘   │
│                                │                                  │
│   ┌──────────────────────────────────────────────────────────┐   │
│   │                基础层 (Infrastructure)                   │   │
│   ├──────────────────────────────────────────────────────────┤   │
│   │  MySQL  │  Redis  │  MongoDB  │  RabbitMQ  │ MinIO  │   │
│   └──────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────┘
```

3.2 核心服务划分

服务名称	端口	职责	依赖服务
member-service	8004	会员管理	MySQL, Redis
points-service	8008	积分管理	MySQL, Redis
coupon-service	8009	优惠券管理	MySQL, Redis
activity-service	8010	活动管理	MySQL, Redis, RabbitMQ
group-buy-service	8011	团购管理	MySQL, Redis
marketing-statistics-service	8012	营销统计	MySQL, MongoDB, Elasticsearch
rule-engine-service	8013	规则引擎	Redis, MySQL

3.3 规则引擎设计

3.3.1 规则引擎架构

```
┌──────────────────────────────────────────────────────────────────┐
│                          规则引擎架构                            │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│   ┌──────────────┐        ┌──────────────┐                      │
│   │ 规则定义层   │        │ 规则执行层   │                      │
│   │              │        │              │                      │
```

```
│   │  - 规则DSL      │◄──►│  - 规则解析器   │                    │
│   │  - 规则模板      │    │  - 规则执行器   │                    │
│   │  - 规则版本      │    │  - 结果缓存     │                    │
│   └──────────────┘    └──────────────┘                    │
│                                │                            │
│   ┌──────────────────────────────────────────────┐         │
│   │                规则仓库 (Rule Repository)          │     │
│   │  营销规则 │ 积分规则 │ 优惠券规则 │ 会员等级规则          │     │
│   └──────────────────────────────────────────────┘         │
│                                │                            │
│   ┌──────────────────────────────────────────────┐         │
│   │                规则监控 (Rule Monitor)             │     │
│   │  规则执行监控 │ 规则效果分析 │ 规则优化建议            │     │
│   └──────────────────────────────────────────────┘         │
└──────────────────────────────────────────────────────────┘
```

3.3.2 规则类型

规则类型	规则内容	适用场景
会员等级规则	等级升级条件、等级权益	会员管理
积分规则	积分获取规则、积分使用规则	积分管理
优惠券规则	优惠券发放规则、使用规则	优惠券管理
活动规则	活动参与规则、优惠规则	活动管理
营销规则	综合营销策略组合	营销活动

3.3.3 规则DSL示例

```yaml
# 会员等级规则示例
name: 会员等级升级规则
version: v1.0
conditions:
  - type: accumulate_consume
    amount: 2000
    unit: CNY
actions:
  - type: upgrade_level
    target_level: silver
    benefits:
```

```yaml
      - discount_rate: 0.95
      - points_multiplier: 1.2

# 积分获取规则示例
name: 消费积分规则
version: v1.0
conditions:
  - type: consume
    amount_range: [0, 1000000]
actions:
  - type: earn_points
    formula: amount * 1
    unit: points

# 优惠券使用规则示例
name: 满减券使用规则
version: v1.0
conditions:
  - type: order_amount
    operator: ">="
    value: 50
actions:
  - type: apply_discount
    discount_type: fixed
    discount_amount: 5
```

四、数据库设计

4.1 会员相关表

4.1.1 会员表

```sql
-- 会员表
CREATE TABLE `member` (
  `id` BIGINT PRIMARY KEY AUTO_INCREMENT,
  `merchant_id` BIGINT NOT NULL COMMENT '商户ID',
  `store_id` BIGINT NOT NULL COMMENT '门店ID',
  `member_no` VARCHAR(50) NOT NULL UNIQUE COMMENT '会员号',
  `member_name` VARCHAR(100) COMMENT '会员姓名',
  `phone` VARCHAR(20) NOT NULL COMMENT '手机号',
```

```sql
  `gender` TINYINT COMMENT '性别 0-未知 1-男 2-女',
  `birthday` DATE COMMENT '生日',
  `address` VARCHAR(500) COMMENT '地址',
  `community_id` BIGINT COMMENT '社区ID',
  `level_id` BIGINT DEFAULT 1 COMMENT '会员等级ID',
  `total_consume` DECIMAL(12,2) DEFAULT 0 COMMENT '累计消费金额',
  `total_points` INT DEFAULT 0 COMMENT '累计积分',
  `available_points` INT DEFAULT 0 COMMENT '可用积分',
  `total_orders` INT DEFAULT 0 COMMENT '累计订单数',
  `register_source` VARCHAR(20) COMMENT '注册来源',
  `referrer_member_id` BIGINT COMMENT '推荐人会员ID',
  `status` TINYINT DEFAULT 1 COMMENT '状态 0-禁用 1-启用',
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  KEY `idx_merchant` (`merchant_id`),
  KEY `idx_store` (`store_id`),
  KEY `idx_phone` (`phone`),
  KEY `idx_community` (`community_id`),
  KEY `idx_level` (`level_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='会员表';
```

4.1.2 会员等级表

```sql
-- 会员等级表
CREATE TABLE `member_level` (
  `id` BIGINT PRIMARY KEY AUTO_INCREMENT,
  `merchant_id` BIGINT NOT NULL,
  `level_name` VARCHAR(50) NOT NULL COMMENT '等级名称',
  `level_code` VARCHAR(20) NOT NULL COMMENT '等级编码',
  `level_order` INT NOT NULL COMMENT '等级排序',
  `upgrade_condition` DECIMAL(12,2) NOT NULL COMMENT '升级条件（累计消费）',
  `discount_rate` DECIMAL(3,2) DEFAULT 1.00 COMMENT '折扣率',
  `points_multiplier` DECIMAL(3,2) DEFAULT 1.00 COMMENT '积分倍数',
  `benefits` TEXT COMMENT '权益描述',
  `status` TINYINT DEFAULT 1,
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  KEY `idx_merchant` (`merchant_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='会员等级表';

-- 初始化会员等级
```

```sql
INSERT INTO `member_level` (`merchant_id`, `level_name`, `level_code`, `level_order`, `upgrade_condition`, `discount_rate`, `points_multiplier`, `benefits`) VALUES
(1, '普通会员', 'NORMAL', 1, 0, 0.98, 1.0, '9.8折优惠'),
(1, '银卡会员', 'SILVER', 2, 500, 0.95, 1.2, '9.5折优惠、生日礼品'),
(1, '金卡会员', 'GOLD', 3, 2000, 0.90, 1.5, '9折优惠、专属客服'),
(1, '钻石会员', 'DIAMOND', 4, 10000, 0.85, 2.0, '8.5折优惠、免费配送、专属商品');
```

4.1.3 会员卡表

```sql
-- 会员卡表
CREATE TABLE `member_card` (
  `id` BIGINT PRIMARY KEY AUTO_INCREMENT,
  `member_id` BIGINT NOT NULL,
  `card_no` VARCHAR(50) NOT NULL UNIQUE COMMENT '卡号',
  `card_type` TINYINT NOT NULL COMMENT '卡类型 1-NFC卡 2-电子卡 3-实体卡',
  `card_status` TINYINT DEFAULT 1 COMMENT '卡状态 0-挂失 1-正常 2-注销',
  `issue_date` DATE COMMENT '发卡日期',
  `expire_date` DATE COMMENT '过期日期',
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  KEY `idx_member` (`member_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='会员卡表';
```

4.2 积分相关表

4.2.1 积分流水表

```
-- 积分流水表
CREATE TABLE `points_transaction` (
  `id` BIGINT PRIMARY KEY AUTO_INCREMENT,
  `merchant_id` BIGINT NOT NULL,
  `store_id` BIGINT NOT NULL,
  `member_id` BIGINT NOT NULL,
  `transaction_no` VARCHAR(50) NOT NULL UNIQUE COMMENT '流水号',
  `transaction_type` TINYINT NOT NULL COMMENT '交易类型 1-获取 2-使用 3-过期 4-调整',
  `points` INT NOT NULL COMMENT '积分数量（正为获取，负为使用）',
```

```sql
  `balance` INT NOT NULL COMMENT '交易后余额',
  `source_type` TINYINT COMMENT '来源类型 1-消费 2-签到 3-活动 4-推荐 5-兑换 6-退款',
  `source_id` BIGINT COMMENT '来源ID（订单ID/活动ID等）',
  `description` VARCHAR(200) COMMENT '描述',
  `operator` VARCHAR(50) COMMENT '操作人',
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  KEY `idx_merchant` (`merchant_id`),
  KEY `idx_store` (`store_id`),
  KEY `idx_member` (`member_id`),
  KEY `idx_type` (`transaction_type`),
  KEY `idx_created` (`created_at`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='积分流水表';
```

4.2.2 积分规则表

```sql
-- 积分规则表
CREATE TABLE `points_rule` (
  `id` BIGINT PRIMARY KEY AUTO_INCREMENT,
  `merchant_id` BIGINT NOT NULL,
  `rule_name` VARCHAR(100) NOT NULL COMMENT '规则名称',
  `rule_type` TINYINT NOT NULL COMMENT '规则类型 1-获取 2-使用',
  `rule_config` TEXT NOT NULL COMMENT '规则配置（JSON）',
  `rule_priority` INT DEFAULT 0 COMMENT '优先级',
  `start_time` TIMESTAMP COMMENT '生效时间',
  `end_time` TIMESTAMP COMMENT '失效时间',
  `status` TINYINT DEFAULT 1,
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  KEY `idx_merchant` (`merchant_id`),
  KEY `idx_type` (`rule_type`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='积分规则表';

-- 积分规则示例
INSERT INTO `points_rule` (`merchant_id`, `rule_name`, `rule_type`, `rule_config`, `rule_priority`) VALUES
(1, '消费积分规则', 1, '{"type":"consume","rate":1,"unit":"CNY"}', 100),
(1, '每日签到积分', 1, '{"type":"sign_in","points":5,"max_per_day":1}', 90),
(1, '活动积分规则', 1, '{"type":"activity","points_range":[10,50]}', 80),
(1, '推荐奖励积分', 1, '{"type":"referral","points":100}', 70),
(1, '积分抵扣规则', 2, '{"type":"deduct","rate":100,"max_ratio":0.3}', 100);
```

4.3 优惠券相关表

4.3.1 优惠券模板表

```sql
-- 优惠券模板表
CREATE TABLE `coupon_template` (
  `id` BIGINT PRIMARY KEY AUTO_INCREMENT,
  `merchant_id` BIGINT NOT NULL,
  `template_name` VARCHAR(100) NOT NULL COMMENT '模板名称',
  `coupon_type` TINYINT NOT NULL COMMENT '优惠券类型 1-满减券 2-折扣券 3-免运费券 4-商品券',
  `coupon_config` TEXT NOT NULL COMMENT '优惠券配置（JSON）',
  `total_quantity` INT NOT NULL COMMENT '发行总量',
  `issued_quantity` INT DEFAULT 0 COMMENT '已发放数量',
  `used_quantity` INT DEFAULT 0 COMMENT '已使用数量',
  `valid_days` INT NOT NULL COMMENT '有效期（天）',
  `start_time` TIMESTAMP COMMENT '生效时间',
  `end_time` TIMESTAMP COMMENT '失效时间',
  `description` VARCHAR(500) COMMENT '使用说明',
  `status` TINYINT DEFAULT 1,
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  KEY `idx_merchant` (`merchant_id`),
  KEY `idx_type` (`coupon_type`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='优惠券模板表';

-- 优惠券模板示例
INSERT INTO `coupon_template` (`merchant_id`, `template_name`, `coupon_type`, `coupon_config`, `total_quantity`, `valid_days`, `description`) VALUES
(1, '满50减5', 1, '{"min_amount":50,"discount_amount":5}', 10000, 30, '满50元减5元'),
(1, '满100减15', 1, '{"min_amount":100,"discount_amount":15}', 10000, 30, '满100元减15元'),
(1, '满200减50', 1, '{"min_amount":200,"discount_amount":50}', 10000, 30, '满200元减50元'),
(1, '8折券', 2, '{"discount_rate":0.8,"max_discount":50}', 10000, 30, '全场8折，最高抵扣50元'),
(1, '免运费券', 3, '{}', 10000, 30, '免配送费');
```

4.3.2 优惠券实例表

```sql
-- 优惠券实例表
CREATE TABLE `coupon` (
  `id` BIGINT PRIMARY KEY AUTO_INCREMENT,
  `merchant_id` BIGINT NOT NULL,
  `template_id` BIGINT NOT NULL COMMENT '模板ID',
  `coupon_code` VARCHAR(50) NOT NULL UNIQUE COMMENT '优惠券码',
  `member_id` BIGINT COMMENT '会员ID（NULL表示未领取）',
  `coupon_status` TINYINT DEFAULT 0 COMMENT '优惠券状态 0-未领取 1-已领取 2-已使用 3-已过期 4-已核销',
  `issue_source` TINYINT COMMENT '发放来源 1-主动发放 2-活动发放 3-领券中心 4-分享',
  `issue_time` TIMESTAMP COMMENT '发放时间',
  `used_time` TIMESTAMP COMMENT '使用时间',
  `order_id` BIGINT COMMENT '订单ID',
  `valid_start_time` TIMESTAMP NOT NULL COMMENT '有效期开始',
  `valid_end_time` TIMESTAMP NOT NULL COMMENT '有效期结束',
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  KEY `idx_merchant` (`merchant_id`),
  KEY `idx_template` (`template_id`),
  KEY `idx_member` (`member_id`),
  KEY `idx_code` (`coupon_code`),
  KEY `idx_status` (`coupon_status`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='优惠券实例表';
```

4.4 活动相关表

4.4.1 营销活动表

```sql
-- 营销活动表
CREATE TABLE `marketing_activity` (
  `id` BIGINT PRIMARY KEY AUTO_INCREMENT,
  `merchant_id` BIGINT NOT NULL,
  `activity_name` VARCHAR(100) NOT NULL COMMENT '活动名称',
  `activity_type` TINYINT NOT NULL COMMENT '活动类型 1-新人专享 2-节日促销 3-会员日 4-社区团购 5-其他',
  `activity_config` TEXT NOT NULL COMMENT '活动配置（JSON）',
  `activity_image` VARCHAR(500) COMMENT '活动图片',
  `activity_desc` VARCHAR(500) COMMENT '活动描述',
  `start_time` TIMESTAMP NOT NULL COMMENT '活动开始时间',
  `end_time` TIMESTAMP NOT NULL COMMENT '活动结束时间',
  `target_type` TINYINT COMMENT '目标类型 1-全部会员 2-指定等级 3-指定会员',
```

```sql
  `target_config` TEXT COMMENT '目标配置',
  `budget` DECIMAL(12,2) COMMENT '活动预算',
  `actual_cost` DECIMAL(12,2) DEFAULT 0 COMMENT '实际花费',
  `activity_status` TINYINT DEFAULT 0 COMMENT '活动状态 0-未开始 1-进行中 2-已结束 3-已取消',
  `created_by` VARCHAR(50),
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  KEY `idx_merchant` (`merchant_id`),
  KEY `idx_type` (`activity_type`),
  KEY `idx_status` (`activity_status`),
  KEY `idx_time` (`start_time`, `end_time`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='营销活动表';

-- 活动配置示例
-- 新人活动配置示例
INSERT INTO `marketing_activity` (`merchant_id`, `activity_name`, `activity_type`, `activity_config`, `start_time`, `end_time`, `target_type`) VALUES
(1, '新人专享礼包', 1, '{
 "new_member_benefit": {
  "coupons": [1, 2],
  "points": 100,
  "discount_rate": 0.9,
  "valid_days": 30
 }
}', '2026-03-01 00:00:00', '2026-12-31 23:59:59', 1);

-- 节日活动配置示例
INSERT INTO `marketing_activity` (`merchant_id`, `activity_name`, `activity_type`, `activity_config`, `start_time`, `end_time`, `target_type`) VALUES
(1, '春节促销', 2, '{
 "promotion_rules": [
  {
   "type": "discount",
   "discount_rate": 0.9,
   "condition": "all"
  },
  {
   "type": "coupon",
   "coupon_template_id": 1,
   "auto_issue": true
  },
  {
```

```sql
    "type": "points_multiplier",
    "multiplier": 2.0
  }
 ]
}', '2026-01-20 00:00:00', '2026-02-10 23:59:59', 1);
```

4.4.2 活动参与记录表

```sql
-- 活动参与记录表
CREATE TABLE `activity_participation` (
  `id` BIGINT PRIMARY KEY AUTO_INCREMENT,
  `merchant_id` BIGINT NOT NULL,
  `activity_id` BIGINT NOT NULL,
  `member_id` BIGINT NOT NULL,
  `participation_type` TINYINT NOT NULL COMMENT '参与类型 1-报名 2-签到 3-完成',
  `participation_time` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  `reward_type` TINYINT COMMENT '奖励类型 1-优惠券 2-积分 3-商品',
  `reward_id` BIGINT COMMENT '奖励ID',
  `reward_quantity` INT COMMENT '奖励数量',
  `remark` VARCHAR(200) COMMENT '备注',
  KEY `idx_activity` (`activity_id`),
  KEY `idx_member` (`member_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='活动参与记录表';
```

4.5 团购相关表

4.5.1 社区团长表

```sql
-- 社区团长表
CREATE TABLE `community_leader` (
  `id` BIGINT PRIMARY KEY AUTO_INCREMENT,
  `merchant_id` BIGINT NOT NULL,
  `member_id` BIGINT NOT NULL,
  `community_id` BIGINT NOT NULL COMMENT '社区ID',
  `leader_name` VARCHAR(100) COMMENT '团长姓名',
  `leader_phone` VARCHAR(20) COMMENT '团长电话',
  `leader_level` TINYINT DEFAULT 1 COMMENT '团长等级',
  `commission_rate` DECIMAL(5,4) DEFAULT 0.0500 COMMENT '佣金比例',
```

```sql
  `total_commission` DECIMAL(12,2) DEFAULT 0 COMMENT '累计佣金',
  `available_commission` DECIMAL(12,2) DEFAULT 0 COMMENT '可提现佣金',
  `total_sales` DECIMAL(12,2) DEFAULT 0 COMMENT '累计销售额',
  `total_orders` INT DEFAULT 0 COMMENT '累计订单数',
  `status` TINYINT DEFAULT 1 COMMENT '状态 0-禁用 1-启用',
  `certified_at` TIMESTAMP COMMENT '认证时间',
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  KEY `idx_merchant` (`merchant_id`),
  KEY `idx_member` (`member_id`),
  KEY `idx_community` (`community_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='社区团长表';
```

4.5.2 团购活动表

```sql
-- 团购活动表
CREATE TABLE `group_buy_activity` (
  `id` BIGINT PRIMARY KEY AUTO_INCREMENT,
  `merchant_id` BIGINT NOT NULL,
  `store_id` BIGINT NOT NULL,
  `leader_id` BIGINT NOT NULL COMMENT '团长ID',
  `activity_name` VARCHAR(100) NOT NULL COMMENT '团购活动名称',
  `activity_config` TEXT NOT NULL COMMENT '团购配置（JSON）',
  `group_price_rule` TEXT NOT NULL COMMENT '团购价规则（JSON）',
  `start_time` TIMESTAMP NOT NULL COMMENT '开始时间',
  `end_time` TIMESTAMP NOT NULL COMMENT '结束时间',
  `delivery_time` TIMESTAMP COMMENT '配送时间',
  `min_participants` INT DEFAULT 10 COMMENT '最少参与人数',
  `max_participants` INT COMMENT '最多参与人数',
  `current_participants` INT DEFAULT 0 COMMENT '当前参与人数',
  `total_amount` DECIMAL(12,2) DEFAULT 0 COMMENT '团购总金额',
  `total_orders` INT DEFAULT 0 COMMENT '团购订单数',
  `activity_status` TINYINT DEFAULT 0 COMMENT '活动状态 0-未开始 1-进行中 2-已
结束 3-已取消',
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  KEY `idx_merchant` (`merchant_id`),
  KEY `idx_store` (`store_id`),
  KEY `idx_leader` (`leader_id`),
  KEY `idx_status` (`activity_status`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='团购活动表';
```

```sql
-- 团购配置示例
INSERT INTO `group_buy_activity` (`merchant_id`, `store_id`, `leader_id`, `activity_name`, `activity_config`, `group_price_rule`, `start_time`, `end_time`, `min_participants`) VALUES
(1, 1, 1, '社区鸡蛋团购', '{
 "products": [
  {"product_id": 1001, "unit_price": 5.0},
  {"product_id": 1002, "unit_price": 3.5}
 ]
}', '{
 "type": "tiered_pricing",
 "tiers": [
  {"min_participants": 10, "discount_rate": 0.95},
  {"min_participants": 20, "discount_rate": 0.90},
  {"min_participants": 50, "discount_rate": 0.85}
 ]
}', '2026-03-10 00:00:00', '2026-03-15 23:59:59', 10);
```

五、核心功能实现

5.1 会员等级自动升级

5.1.1 业务流程

```
会员消费 → 累计消费金额增加 → 触发等级升级检查 → 符合升级条件 → 自动升级会员等级 → 发送升级通知 → 应用新等级权益
```

5.1.2 核心代码实现

```java
/**
 * 会员等级升级服务
 */
@Service
@Slf4j
public class MemberLevelUpgradeService {
```

```java
    @Autowired
    private MemberRepository memberRepository;

    @Autowired
    private MemberLevelRepository levelRepository;

    @Autowired
    private PointsTransactionService pointsTransactionService;

    @Autowired
    private NotificationService notificationService;

    /**
  * 消费后检查并升级会员等级
  */
    @Transactional(rollbackFor = Exception.class)
    public void checkAndUpgradeLevel(Long memberId, BigDecimal consumeAmount) {
        // 1. 查询会员信息
        Member member = memberRepository.findById(memberId)
            .orElseThrow(() -> new BusinessException("会员不存在"));

        // 2. 更新累计消费金额

member.setTotalConsume(member.getTotalConsume().add(consumeAmount));
        memberRepository.save(member);

        // 3. 查询会员等级规则
        List<MemberLevel> levels =
levelRepository.findByMerchantIdOrderByLevelOrder(
            member.getMerchantId()
        );

        // 4. 检查是否满足升级条件
        for (MemberLevel level : levels) {
            if (level.getLevelOrder() >
member.getLevel().getLevelOrder() &&

member.getTotalConsume().compareTo(level.getUpgradeCondition()) >= 0) {

                // 5. 执行升级
                upgradeLevel(member, level);
                break;
            }
        }
    }
```

```java
    /**
     * 执行会员升级
     */
    private void upgradeLevel(Member member, MemberLevel newLevel) {
        Long oldLevelId = member.getLevelId();
        Long newLevelId = newLevel.getId();

        // 1. 更新会员等级
        member.setLevelId(newLevelId);
        memberRepository.save(member);

        // 2. 发放升级奖励
        issueUpgradeReward(member, newLevel);

        // 3. 发送升级通知
        sendUpgradeNotification(member, oldLevelId, newLevelId);

        log.info("会员{}升级成功，从{}升级到{}", member.getMemberNo(),
oldLevelId, newLevelId);
    }

    /**
     * 发放升级奖励
     */
    private void issueUpgradeReward(Member member, MemberLevel newLevel) {
        // 根据等级配置发放升级奖励（优惠券、积分等）
        // 示例：发放升级优惠券
        // ...
    }

    /**
     * 发送升级通知
     */
    private void sendUpgradeNotification(Member member, Long oldLevelId, Long newLevelId)
{
        // 发送小程序推送/短信通知
        notificationService.sendLevelUpgradeNotification(member,
oldLevelId, newLevelId);
    }
}
```

5.2 积分自动累计与使用

5.2.1 积分获取规则引擎

```java
/**
 * 积分规则引擎
 */
@Service
@Slf4j
public class PointsRuleEngine {

    @Autowired
    private PointsRuleRepository ruleRepository;

    @Autowired
    private PointsTransactionService transactionService;

    /**
  * 执行积分获取规则
  */
    public void executeEarnRule(Long memberId, Long orderId, BigDecimal orderAmount) {
        // 1. 查询有效的积分获取规则
        List<PointsRule> rules = ruleRepository.findEarnRules(
            member.getMerchantId(),
            LocalDateTime.now()
        );

        // 2. 按优先级排序

rules.sort(Comparator.comparing(PointsRule::getRulePriority).reversed()
);

        // 3. 依次执行规则
        for (PointsRule rule : rules) {
            try {
                executeRule(memberId, orderId, orderAmount, rule);
            } catch (Exception e) {
                log.error("执行积分规则失败: ruleId={}", rule.getId(), e);
            }
        }
    }

    /**
  * 执行单条规则
  */
    private void
executeRule(Long memberId, Long orderId, BigDecimal orderAmount, PointsRule rule) {
        // 解析规则配置
        JSONObject config = JSON.parseObject(rule.getRuleConfig());
```

```java
            // 根据规则类型执行
            String ruleType = config.getString("type");

            switch (ruleType) {
                case "consume":
                    // 消费积分规则
                    int points = calculateConsumePoints(orderAmount, config);
                    transactionService.earnPoints(memberId, points, "消费获取积分", orderId);
                    break;

                case "sign_in":
                    // 签到积分规则（在签到服务中调用）
                    break;

                case "activity":
                    // 活动积分规则（在活动服务中调用）
                    break;

                case "referral":
                    // 推荐奖励积分规则（在推荐服务中调用）
                    break;

                default:
                    log.warn("未知的积分规则类型:{}", ruleType);
            }
        }

        /**
     * 计算消费积分
     */
        private int calculateConsumePoints(BigDecimal orderAmount, JSONObject config) {
            BigDecimal rate = config.getBigDecimal("rate");
            return orderAmount.multiply(rate).intValue();
        }
    }
```

5.2.2 积分使用（抵扣）

```
    /**
     * 积分使用服务
     */
```

```java
@Service
@Slf4j
public class PointsDeductionService {

    @Autowired
    private MemberRepository memberRepository;

    @Autowired
    private PointsRuleRepository ruleRepository;

    @Autowired
    private PointsTransactionService transactionService;

    /**
  * 计算最大可用积分抵扣金额
  */
    public BigDecimal calculateMaxDeduction(Long memberId, BigDecimal orderAmount) {
        // 1. 查询会员可用积分
        Member member = memberRepository.findById(memberId)
            .orElseThrow(() -> new BusinessException("会员不存在"));

        int availablePoints = member.getAvailablePoints();

        // 2. 查询积分使用规则
        PointsRule rule =
ruleRepository.findUseRule(member.getMerchantId(), LocalDateTime.now())
            .orElseThrow(() -> new BusinessException("未配置积分使用规则"));

        JSONObject config = JSON.parseObject(rule.getRuleConfig());

        // 3. 计算最大抵扣金额
        BigDecimal pointsValue = BigDecimal.valueOf(availablePoints)
            .divide(config.getBigDecimal("rate"), 2, RoundingMode.DOWN);

        // 4. 应用最大抵扣比例限制
        BigDecimal maxDeductionRatio = config.getBigDecimal("max_ratio");
        BigDecimal maxDeductionAmount =
orderAmount.multiply(maxDeductionRatio);

        return pointsValue.min(maxDeductionAmount);
    }

    /**
  * 使用积分抵扣
  */
    @Transactional(rollbackFor = Exception.class)
```

```java
    public void deductPoints(Long memberId, int deductPoints, Long orderId, String description)
{
        // 1. 扣除积分
        transactionService.usePoints(memberId, deductPoints,
description, orderId);

        log.info("会员{}使用{}积分抵扣订单{}", memberId, deductPoints,
orderId);
    }
}
```

5.3 优惠券自动发放与核销

5.3.1 优惠券发放

```java
/**
 * 优惠券发放服务
 */
@Service
@Slf4j
public class CouponIssueService {

    @Autowired
    private CouponTemplateRepository templateRepository;

    @Autowired
    private CouponRepository couponRepository;

    /**
  * 发放优惠券
  */
    @Transactional(rollbackFor = Exception.class)
    public void issueCoupon(Long templateId, Long memberId, IssueSource source) {
        // 1. 查询优惠券模板
        CouponTemplate template = templateRepository.findById(templateId)
            .orElseThrow(() -> new BusinessException("优惠券模板不存在"));

        // 2. 检查发放数量
        if (template.getIssuedQuantity() >= template.getTotalQuantity())
{
            throw new BusinessException("优惠券已发放完毕");
        }
```

```java
        // 3. 生成优惠券码
        String couponCode = generateCouponCode();

        // 4. 创建优惠券实例
        Coupon coupon = new Coupon();
        coupon.setMerchantId(template.getMerchantId());
        coupon.setTemplateId(templateId);
        coupon.setCouponCode(couponCode);
        coupon.setMemberId(memberId);
        coupon.setCouponStatus(CouponStatus.ISSUED);
        coupon.setIssueSource(source.getValue());
        coupon.setIssueTime(LocalDateTime.now());

        // 计算有效期
        LocalDateTime validStartTime = template.getStartTime() != null
            ? template.getStartTime()
            : LocalDateTime.now();
        LocalDateTime validEndTime = template.getEndTime() != null
            ? template.getEndTime()
            : LocalDateTime.now().plusDays(template.getValidDays());

        coupon.setValidStartTime(validStartTime);
        coupon.setValidEndTime(validEndTime);

        // 5. 保存优惠券
        couponRepository.save(coupon);

        // 6. 更新模板发放数量
        template.setIssuedQuantity(template.getIssuedQuantity() + 1);
        templateRepository.save(template);

        log.info("成功发放优惠券: templateId={}, memberId={}", templateId,
memberId);
    }

    /**
  * 生成优惠券码
  */
    private String generateCouponCode() {
        return "CP" + System.currentTimeMillis() +
RandomStringUtils.randomNumeric(6);
    }

    /**
  * 批量发放优惠券
  */
    @Transactional(rollbackFor = Exception.class)
```

```java
    public void batchIssueCoupon(Long templateId, List<Long> memberIds, IssueSource source)
{
        for (Long memberId : memberIds) {
            try {
                issueCoupon(templateId, memberId, source);
            } catch (Exception e) {
                log.error("发放优惠券失败: memberId={}", memberId, e);
            }
        }
    }
}
```

5.3.2 优惠券核销

```
/**
 * 优惠券核销服务
 */
@Service
@Slf4j
public class CouponUseService {

    @Autowired
    private CouponRepository couponRepository;

    @Autowired
    private OrderRepository orderRepository;

    /**
  * 使用优惠券
  */
    @Transactional(rollbackFor = Exception.class)
    public BigDecimal useCoupon(String couponCode, Long memberId, BigDecimal
orderAmount) {
        // 1. 查询优惠券
        Coupon coupon =
couponRepository.findByCouponCodeAndMemberId(couponCode, memberId)
            .orElseThrow(() -> new BusinessException("优惠券不存在或不属于该会
员"));

        // 2. 验证优惠券状态
        if (coupon.getCouponStatus() != CouponStatus.ISSUED) {
            throw new BusinessException("优惠券已使用或已过期");
        }
```

```java
        // 3. 验证有效期
        LocalDateTime now = LocalDateTime.now();
        if (now.isBefore(coupon.getValidStartTime()) ||
now.isAfter(coupon.getValidEndTime())) {
            throw new BusinessException("优惠券不在有效期内");
        }

        // 4. 查询优惠券模板
        CouponTemplate template =
couponRepository.findById(coupon.getTemplateId())
            .orElseThrow(() -> new BusinessException("优惠券模板不存在"));

        // 5. 验证使用条件
        validateCouponCondition(template, orderAmount);

        // 6. 计算优惠金额
        BigDecimal discountAmount = calculateDiscountAmount(template,
orderAmount);

        // 7. 标记优惠券已使用（在订单支付成功后调用）
        // coupon.setCouponStatus(CouponStatus.USED);
        // coupon.setUsedTime(now);
        // couponRepository.save(coupon);

        // 8. 更新模板使用数量
        template.setUsedQuantity(template.getUsedQuantity() + 1);
        templateRepository.save(template);

        return discountAmount;
    }

    /**
  * 验证优惠券使用条件
  */
    private void validateCouponCondition(CouponTemplate template, BigDecimal orderAmount)
{
        JSONObject config = JSON.parseObject(template.getCouponConfig());

        // 满减券验证
        if (template.getCouponType() == CouponType.DISCOUNT_AMOUNT) {
            BigDecimal minAmount = config.getBigDecimal("min_amount");
            if (orderAmount.compareTo(minAmount) < 0) {
                throw new BusinessException("订单金额不满足使用条件");
            }
        }
```

```java
    }

    /**
     * 计算优惠金额
     */
    private BigDecimal calculateDiscountAmount(CouponTemplate template, BigDecimal orderAmount) {
        JSONObject config = JSON.parseObject(template.getCouponConfig());

        switch (template.getCouponType()) {
            case DISCOUNT_AMOUNT: // 满减券
                return config.getBigDecimal("discount_amount");

            case DISCOUNT_RATE: // 折扣券
                BigDecimal discountRate = config.getBigDecimal("discount_rate");
                BigDecimal discount = orderAmount.multiply(BigDecimal.ONE.subtract(discountRate));
                BigDecimal maxDiscount = config.getBigDecimal("max_discount");
                return discount.min(maxDiscount);

            case FREE_DELIVERY: // 免运费券
                return config.getBigDecimal("delivery_fee");

            case PRODUCT_COUPON: // 商品券
                return config.getBigDecimal("discount_amount");

            default:
                return BigDecimal.ZERO;
        }
    }
}
```

5.4 营销活动参与

5.4.1 活动参与

```java
/**
 * 营销活动参与服务
 */
@Service
@Slf4j
public class ActivityParticipationService {
```

```java
    @Autowired
    private MarketingActivityRepository activityRepository;

    @Autowired
    private ActivityParticipationRepository participationRepository;

    @Autowired
    private CouponIssueService couponIssueService;

    @Autowired
    private PointsTransactionService pointsTransactionService;

    /**
  * 参与营销活动
  */
    @Transactional(rollbackFor = Exception.class)
    public void participateActivity(Long activityId, Long memberId) {
        // 1. 查询活动
        MarketingActivity activity = activityRepository.findById(activityId)
            .orElseThrow(() -> new BusinessException("活动不存在"));

        // 2. 验证活动状态
        if (activity.getActivityStatus() != ActivityStatus.IN_PROGRESS)
{
            throw new BusinessException("活动未开始或已结束");
        }

        // 3. 验证活动时间
        LocalDateTime now = LocalDateTime.now();
        if (now.isBefore(activity.getStartTime()) ||
now.isAfter(activity.getEndTime())) {
            throw new BusinessException("不在活动时间内");
        }

        // 4. 验证目标人群
        if (!validateTargetUser(activity, memberId)) {
            throw new BusinessException("不符合活动参与条件");
        }

        // 5. 检查是否已参与
        if (hasParticipated(activityId, memberId)) {
            throw new BusinessException("已参与过该活动");
        }

        // 6. 执行活动规则
        executeActivityRule(activity, memberId);
```

```
        // 7. 记录参与
        recordParticipation(activity, memberId);

        log.info("会员{}成功参与活动{}", memberId, activityId);
    }

    /**
  * 验证目标人群
  */
    private boolean validateTargetUser(MarketingActivity activity, Long memberId) {
        JSONObject config = JSON.parseObject(activity.getTargetConfig());

        Integer targetType = activity.getTargetType();

        switch (targetType) {
            case 1: // 全部会员
                return true;

            case 2: // 指定等级
                // 验证会员等级是否符合
                return true;

            case 3: // 指定会员
                List<Long> targetMemberIds =
config.getJSONArray("member_ids")
                    .toJavaList(Long.class);
                return targetMemberIds.contains(memberId);

            default:
                return false;
        }
    }

    /**
  * 执行活动规则
  */
    private void executeActivityRule(MarketingActivity activity, Long memberId) {
        JSONObject config =
JSON.parseObject(activity.getActivityConfig());

        // 新人专享礼包
        if (activity.getActivityType() ==
ActivityType.NEW_MEMBER_BENEFIT) {
            JSONObject benefit = config.getJSONObject("new_member_benefit");
```

```java
            // 发放优惠券
            JSONArray coupons = benefit.getJSONArray("coupons");
            for (int i = 0; i < coupons.size(); i++) {
                Long templateId = coupons.getLong(i);
                couponIssueService.issueCoupon(templateId, memberId,
IssueSource.ACTIVITY);
            }

            // 发放积分
            Integer points = benefit.getInteger("points");
            pointsTransactionService.earnPoints(memberId, points, "新人礼
包", null);
        }

        // 其他活动类型...
    }

    /**
  * 记录参与
  */
    private void recordParticipation(MarketingActivity activity, Long memberId) {
        ActivityParticipation participation = new ActivityParticipation();
        participation.setMerchantId(activity.getMerchantId());
        participation.setActivityId(activity.getId());
        participation.setMemberId(memberId);
        participation.setParticipationType(ParticipationType.COMPLETE);
        participation.setParticipationTime(LocalDateTime.now());

        participationRepository.save(participation);
    }
}
```

六、规则引擎设计

6.1 规则引擎架构

```
┌──────────────────────────────────────────────────────────────────────────┐
│                                 规则引擎架构设计                                 │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                                │
│   ┌──────────────────┐          ┌──────────────────┐                        │
```

```
│   │  规则定义      │        │  规则存储      │                        │
│   │  - DSL定义     │◀──────▶│  - 规则库      │                        │
│   │  - 规则模板    │        │  - 版本管理    │                        │
│   └────────────────┘        └────────────────┘                        │
│                                    │                                   │
│   ┌──────────────────────────────────────────────────────────┐       │
│   │                    规则引擎核心                          │       │
│   ├──────────────────────────────────────────────────────────┤       │
│   │   ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  │       │
│   │   │ 规则解析器   │  │ 规则编译器   │  │ 规则执行器   │  │       │
│   │   └──────────────┘  └──────────────┘  └──────────────┘  │       │
│   │   ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  │       │
│   │   │ 上下文管理   │  │ 结果缓存     │  │ 监控统计     │  │       │
│   │   └──────────────┘  └──────────────┘  └──────────────┘  │       │
│   └──────────────────────────────────────────────────────────┘       │
│                                    │                                   │
│   ┌──────────────────────────────────────────────────────────┐       │
│   │                    规则接口层                            │       │
│   ├──────────────────────────────────────────────────────────┤       │
│   │  会员等级规则API │ 积分规则API │ 优惠券规则API          │       │
│   └──────────────────────────────────────────────────────────┘       │
└──────────────────────────────────────────────────────────────────────┘
```

6.2 规则DSL设计

6.2.1 规则DSL语法

```
规则DSL语法结构:
rule <规则名称>
  version <规则版本>
  priority <优先级>

  when <条件列表> then
    <动作列表>
  end

  when <条件列表> then
    <动作列表>
  end
end

条件类型:
- member_level: 会员等级
- total_consume: 累计消费金额
```

```
- order_amount: 订单金额
- community: 社区
- time: 时间

动作类型:
- upgrade_level: 升级会员等级
- earn_points: 获取积分
- deduct_points: 扣除积分
- issue_coupon: 发放优惠券
- apply_discount: 应用折扣
```

6.2.2 规则示例

```yaml
# 会员等级升级规则
rule: 会员等级升级规则
version: v1.0
priority: 100

when:
  - condition: total_consume
    operator: ">="
    value: 500
  - condition: member_level
    operator: "<"
    value: 2
then:
  - action: upgrade_level
    target_level: 2
  - action: issue_coupon
    template_id: 10
  - action: send_notification
    type: level_upgrade
end

# 积分获取规则
rule: 消费积分获取规则
version: v1.0
priority: 100

when:
  - condition: order_amount
    operator: ">"
    value: 0
```

```yaml
  then:
    - action: earn_points
      formula: order_amount * 1
      unit: points
  end

  # 优惠券自动发放规则
  rule: 新人优惠券发放规则
  version: v1.0
  priority: 100

  when:
    - condition: member_created_days
      operator: "<="
      value: 1
  then:
    - action: issue_coupon
      template_id: 1
      quantity: 1
    - action: issue_coupon
      template_id: 2
      quantity: 1
  end
```

6.3 规则引擎实现

```java
/**
 * 营销规则引擎
 */
@Component
@Slf4j
public class MarketingRuleEngine {

    @Autowired
    private RuleRepository ruleRepository;

    @Autowired
    private RuleCache ruleCache;

    /**
  * 执行规则
  */
    public RuleExecutionResult executeRules(String ruleType, RuleContext context) {
```

```java
        // 1. 查询规则
        List<Rule> rules = ruleCache.getRules(ruleType);

        if (CollectionUtils.isEmpty(rules)) {
            return RuleExecutionResult.empty();
        }

        // 2. 按优先级排序
        rules.sort(Comparator.comparing(Rule::getPriority).reversed());

        // 3. 执行规则
        RuleExecutionResult result = new RuleExecutionResult();
        for (Rule rule : rules) {
            try {
                executeRule(rule, context, result);
            } catch (Exception e) {
                log.error("执行规则失败: ruleId={}", rule.getId(), e);
            }
        }

        return result;
    }

    /**
    * 执行单条规则
    */
    private void executeRule(Rule rule, RuleContext context, RuleExecutionResult result) {
        // 解析规则配置
        JSONObject config = JSON.parseObject(rule.getRuleConfig());

        JSONArray whenClauses = config.getJSONArray("when");
        JSONArray thenClauses = config.getJSONArray("then");

        // 评估when条件
        boolean conditionMet = evaluateConditions(whenClauses, context);

        if (conditionMet) {
            // 执行then动作
            executeActions(thenClauses, context, result);

            result.addExecutedRule(rule.getId());
        }
    }

    /**
    * 评估条件
```

```
 */
    private boolean evaluateConditions(JSONArray conditions, RuleContext context) {
        for (int i = 0; i < conditions.size(); i++) {
            JSONObject condition = conditions.getJSONObject(i);

            String conditionType = condition.getString("condition");
            String operator = condition.getString("operator");
            Object value = condition.get("value");

            if (!evaluateCondition(conditionType, operator, value,
context)) {
                return false;
            }
        }

        return true;
    }

    /**
 * 评估单个条件
 */
    private boolean evaluateCondition(String conditionType, String operator, Object value,
RuleContext context) {
        switch (conditionType) {
            case "total_consume":
                BigDecimal totalConsume =
context.getMember().getTotalConsume();
                BigDecimal threshold = new BigDecimal(value.toString());
                return compare(totalConsume, operator, threshold);

            case "order_amount":
                BigDecimal orderAmount = context.getOrderAmount();
                BigDecimal threshold = new BigDecimal(value.toString());
                return compare(orderAmount, operator, threshold);

            case "member_level":
                Integer memberLevel = context.getMember().getLevelId();
                Integer threshold = Integer.parseInt(value.toString());
                return compare(memberLevel, operator, threshold);

            default:
                return false;
        }
    }

    /**
```

```
 * 执行动作
 */
    private void executeActions(JSONArray actions, RuleContext context, RuleExecutionResult result) {
        for (int i = 0; i < actions.size(); i++) {
            JSONObject action = actions.getJSONObject(i);

            String actionType = action.getString("action");

            switch (actionType) {
                case "upgrade_level":
                    // 升级会员等级
                    executeUpgradeLevelAction(action, context, result);
                    break;

                case "earn_points":
                    // 获取积分
                    executeEarnPointsAction(action, context, result);
                    break;

                case "issue_coupon":
                    // 发放优惠券
                    executeIssueCouponAction(action, context, result);
                    break;

                case "apply_discount":
                    // 应用折扣
                    executeApplyDiscountAction(action, context, result);
                    break;

                default:
                    log.warn("未知的动作类型: {}", actionType);
            }
        }
    }

    /**
 * 比较操作
 */
    private <T extends Comparable<T>> boolean compare(T actual, String operator, T expected) {
        int comparison = actual.compareTo(expected);

        switch (operator) {
            case ">":
```

```java
                return comparison > 0;
            case ">=":
                return comparison >= 0;
            case "<":
                return comparison < 0;
            case "<=":
                return comparison <= 0;
            case "==":
                return comparison == 0;
            case "!=":
                return comparison != 0;
            default:
                return false;
        }
    }
}
```

七、营销活动管理

7.1 活动类型

活动类型	活动描述	适用场景
新人专享	新用户注册福利	拉新
节日促销	节假日主题促销	提升节日销售
会员日	会员专属活动日	提升会员粘性
社区团购	社区团长组织团购	社区运营
签到有礼	每日签到活动	提升日活
秒杀活动	限时秒杀商品	提升转化
满减活动	满减优惠	提升客单价

7.2 活动创建流程

```
活动创建流程:
1. 选择活动类型
```

```
2. 配置活动基本信息（名称、时间、描述）
3. 配置活动规则（优惠规则、参与条件）
4. 配置活动预算
5. 上传活动素材（图片、海报）
6. 预览活动效果
7. 发布活动
8. 活动推广（小程序推送、海报、短信）
9. 活动数据监控
10. 活动总结分析
```

7.3 活动数据统计

```
/**
 * 活动数据统计服务
 */
@Service
@Slf4j
public class ActivityStatisticsService {

    @Autowired
    private ActivityParticipationRepository participationRepository;

    @Autowired
    private OrderRepository orderRepository;

    /**
  * 获取活动统计数据
  */
    public ActivityStatistics getActivityStatistics(Long activityId) {
        ActivityStatistics statistics = new ActivityStatistics();

        // 1. 活动参与人数
        long participationCount =
participationRepository.countByActivityId(activityId);
        statistics.setParticipationCount(participationCount);

        // 2. 活动订单数
        long orderCount = orderRepository.countByActivityId(activityId);
        statistics.setOrderCount(orderCount);

        // 3. 活动销售额
        BigDecimal salesAmount =
orderRepository.sumSalesAmountByActivityId(activityId);
```

```java
        statistics.setSalesAmount(salesAmount);

        // 4. 活动成本
        BigDecimal cost = calculateActivityCost(activityId);
        statistics.setCost(cost);

        // 5. 活动ROI
        BigDecimal roi = calculateROI(salesAmount, cost);
        statistics.setRoi(roi);

        return statistics;
    }

    /**
    * 计算活动成本
    */
    private BigDecimal calculateActivityCost(Long activityId) {
        // 计算活动总成本（优惠券成本、积分成本等）
        return BigDecimal.ZERO;
    }

    /**
    * 计算ROI
    */
    private BigDecimal calculateROI(BigDecimal revenue, BigDecimal cost) {
        if (cost.compareTo(BigDecimal.ZERO) == 0) {
            return BigDecimal.ZERO;
        }

        return revenue.subtract(cost).divide(cost, 2,
RoundingMode.HALF_UP);
    }
}
```

八、数据统计分析

8.1 会员分析

8.1.1 会员增长趋势

```sql
-- 会员增长趋势
SELECT
  DATE(created_at) AS date,
  COUNT(*) AS new_member_count,
  SUM(CASE WHEN level_id = 1 THEN 1 ELSE 0 END) AS normal_member_count,
  SUM(CASE WHEN level_id = 2 THEN 1 ELSE 0 END) AS silver_member_count,
  SUM(CASE WHEN level_id = 3 THEN 1 ELSE 0 END) AS gold_member_count,
  SUM(CASE WHEN level_id = 4 THEN 1 ELSE 0 END) AS diamond_member_count
FROM member
WHERE merchant_id = 1
  AND created_at >= DATE_SUB(NOW(), INTERVAL 30 DAY)
GROUP BY DATE(created_at)
ORDER BY date DESC;
```

8.1.2 会员活跃度分析

```sql
-- 会员活跃度分析
SELECT
  member_level_id,
  COUNT(DISTINCT member_id) AS active_member_count,
  AVG(DATEDIFF(NOW(), MAX(created_at))) AS avg_days_since_last_order
FROM `order`
WHERE merchant_id = 1
  AND created_at >= DATE_SUB(NOW(), INTERVAL 30 DAY)
GROUP BY member_level_id;
```

8.2 营销活动效果分析

8.2.1 活动参与分析

```
-- 活动参与分析
SELECT
  a.activity_name,
  a.activity_type,
  COUNT(DISTINCT ap.member_id) AS participant_count,
  COUNT(DISTINCT o.id) AS order_count,
  SUM(o.payable_amount) AS sales_amount,
  SUM(o.discount_amount) AS discount_amount
```

```sql
FROM marketing_activity a
LEFT JOIN activity_participation ap ON a.id = ap.activity_id
LEFT JOIN `order` o ON ap.member_id = o.member_id
  AND o.created_at BETWEEN a.start_time AND a.end_time
WHERE a.merchant_id = 1
  AND a.activity_status = 2
GROUP BY a.id, a.activity_name, a.activity_type
ORDER BY sales_amount DESC;
```

8.3 社区消费分析

8.3.1 社区消费力分析

```sql
-- 社区消费力分析
SELECT
  m.community_id,
  c.community_name,
  COUNT(DISTINCT m.id) AS member_count,
  COUNT(DISTINCT o.id) AS order_count,
  SUM(o.payable_amount) AS total_sales_amount,
  AVG(o.payable_amount) AS avg_order_amount
FROM member m
LEFT JOIN community c ON m.community_id = c.id
LEFT JOIN `order` o ON m.id = o.member_id
  AND o.created_at >= DATE_SUB(NOW(), INTERVAL 30 DAY)
WHERE m.merchant_id = 1
GROUP BY m.community_id, c.community_name
ORDER BY total_sales_amount DESC;
```

九、前端界面设计

9.1 Web管理端界面

9.1.1 会员管理界面

```
会员管理界面布局:
┌──────────────────────────────────────────────────────────────────────────────┐
```

```
│  海邻到家管理系统 > 会员管理                                    │
├──────────────────────────────────────────────────────────────┤
│  [会员列表] [会员等级] [积分管理] [优惠券] [活动管理]      │
├──────────────────────────────────────────────────────────────┤
│  搜索: [手机号/会员号]   筛选: [会员等级▼] [注册时间]         │
├──────────────────────────────────────────────────────────────┤
│  会员号  姓名  手机号  等级  累计消费  可用积分 操作 │
│  M000001   张三    138****1234 银卡会员 ¥2,500.00  1,200 [详情]│
│  M000002   李四    139****5678 金卡会员 ¥8,500.00  5,600 [详情]│
│  M000003   王五    137****9012 钻石会员¥25,000.00  12,800[详情]│
└──────────────────────────────────────────────────────────────┘
```

9.1.2 营销活动创建界面

营销活动创建界面:

```
┌──────────────────────────────────────────────────────────────┐
│  创建营销活动                                                  │
├──────────────────────────────────────────────────────────────┤
│  基本信息:                                                     │
│  活动名称: [_____________________________]                     │
│  活动类型: [节日促销 ▼]                                        │
│  活动时间: [开始时间] - [结束时间]                             │
│  活动预算: [_________] 元                                      │
├──────────────────────────────────────────────────────────────┤
│  活动规则:                                                     │
│  优惠类型: [满减 ▼]                                            │
│  满减金额: 满[50]元 减[5]元                                    │
│  优惠商品: [全部商品] / [指定商品]                             │
│  参与条件: [全部会员] / [指定等级] / [指定会员]                │
├──────────────────────────────────────────────────────────────┤
│  活动推广:                                                     │
│  □ 小程序推送                                                  │
│  □ 短信通知                                                    │
│  □ 门店海报                                                    │
├──────────────────────────────────────────────────────────────┤
│  活动预览:                                                     │
│  [活动海报预览]                                                │
├──────────────────────────────────────────────────────────────┤
│  [取消] [保存草稿] [立即发布]                                  │
└──────────────────────────────────────────────────────────────┘
```

9.2 小程序端界面

9.2.1 会员中心界面

```
会员中心界面:
┌──────────────────────────────────────────────────────────┐
│  会员中心                                                  │
├──────────────────────────────────────────────────────────┤
│  [会员头像]                                                │
│  钻石会员                                                  │
│  消费 ¥25,000 / 升级还需 ¥0                               │
├──────────────────────────────────────────────────────────┤
│  [我的积分]  [我的优惠券]  [积分兑换]                      │
├──────────────────────────────────────────────────────────┤
│  会员权益:                                                 │
│  • 全场8.5折优惠                                           │
│  • 免费配送服务                                            │
│  • 专属商品优惠                                            │
│  • 专属客服服务                                            │
├──────────────────────────────────────────────────────────┤
│  营销活动:                                                 │
│  [春节促销] [新人专享] [会员日] [社区团购]                 │
└──────────────────────────────────────────────────────────┘
```

9.2.2 优惠券界面

```
优惠券界面:
┌──────────────────────────────────────────────────────────┐
│  我的优惠券                                                │
├──────────────────────────────────────────────────────────┤
│  [未使用(3)] [已使用(5)] [已过期(2)]                        │
├──────────────────────────────────────────────────────────┤
│  满200减50                                                 │
│  有效期至 2026-04-01                                       │
│  [立即使用]                                                │
├──────────────────────────────────────────────────────────┤
│  全场8折                                                   │
│  有效期至 2026-03-31                                       │
│  [立即使用]                                                │
├──────────────────────────────────────────────────────────┤
│  [领券中心]                                                │
└──────────────────────────────────────────────────────────┘
```

十、接口设计

10.1 会员服务接口

10.1.1 会员注册

```yaml
接口名称: 会员注册
请求方式: POST
请求路径: /api/member/register

请求体:
  phone: String（手机号）
  code: String（验证码）
  member_name: String（会员姓名）
  gender: Integer（性别）
  birthday: String（生日）
  address: String（地址）

响应数据:
  member_id: Long
  member_no: String
  available_points: Integer
```

10.1.2 会员查询

```
接口名称: 会员信息查询
请求方式: GET
请求路径: /api/member/info

请求参数:
  member_id: Long

响应数据:
  member_id: Long
  member_no: String
  member_name: String
  phone: String
  level_id: Integer
  level_name: String
  total_consume: BigDecimal
```

```yaml
total_points: Integer
available_points: Integer
total_orders: Integer
```

10.2 积分服务接口

10.2.1 积分流水查询

```yaml
接口名称: 积分流水查询
请求方式: GET
请求路径: /api/points/transactions

请求参数:
  member_id: Long
  page: Integer
  size: Integer

响应数据:
  - id: Long
    transaction_type: Integer
    points: Integer
    balance: Integer
    description: String
    created_at: Timestamp
```

10.3 优惠券服务接口

10.3.1 优惠券列表

```
接口名称: 我的优惠券列表
请求方式: GET
请求路径: /api/coupon/my-coupons

请求参数:
  member_id: Long
  status: Integer (优惠券状态 0-未使用 1-已使用 2-已过期)
  page: Integer
  size: Integer
```

```yaml
响应数据:
  - id: Long
    template_name: String
    coupon_type: Integer
    coupon_config: Object
    valid_start_time: Timestamp
    valid_end_time: Timestamp
    coupon_status: Integer
```

10.4 活动服务接口

10.4.1 活动列表

```yaml
接口名称: 营销活动列表
请求方式: GET
请求路径: /api/activity/list

请求参数:
  activity_type: Integer（活动类型）
  status: Integer（活动状态）

响应数据:
  - id: Long
    activity_name: String
    activity_type: Integer
    activity_image: String
    activity_desc: String
    start_time: Timestamp
    end_time: Timestamp
    activity_status: Integer
```

十一、实施计划

11.1 开发阶段

阶段	任务	工期	交付物
需求分析	社区营销需求梳理	1周	需求文档

阶段	任务	工期	交付物
系统设计	模块设计、规则引擎设计	1.5周	设计文档
后端开发	会员/积分/优惠券/活动/团购服务	4周	后端服务
规则引擎	规则引擎开发	2周	规则引擎
前端开发	Web管理端、小程序端	3周	前端应用
测试验收	功能测试、性能测试	2周	测试报告
上线部署	灰度发布、全量上线	1周	生产环境
总计		**12.5周**	

11.2 里程碑

里程碑	时间节点	交付内容
M1：需求确认	第1周	社区营销需求文档
M2：设计完成	第2.5周	模块设计、规则引擎设计
M3：规则引擎完成	第4.5周	规则引擎上线
M4：后端完成	第8.5周	后端服务全部上线
M5：前端完成	第11.5周	前端应用完成
M6：测试通过	第13.5周	测试报告通过
M7：正式上线	第14.5周	生产环境上线

附录

A. 术语表

术语	解释
会员等级	根据累计消费金额划分的会员层级
积分	会员通过消费、活动等方式获取的虚拟货币
优惠券	可在消费时抵扣金额或享受折扣的凭证
营销活动	为促进销售而开展的促销活动
社区团购	由社区团长组织的团购活动
规则引擎	根据预设规则自动执行营销策略的引擎
NFC会员卡	近场通信会员卡，刷卡识别会员
ROI	投资回报率（Return on Investment）

B. 参考文档

- 《海邻到家智能收银系统总体架构设计文档》
- 《海邻到家品牌规范手册》
- 《社区便利店运营指南》
- 《会员管理最佳实践》

文档变更记录

版本	日期	变更内容	作者
v1.0	2026-03-06	初始版本	系统

版权声明

本文档为海邻到家智能收银系统社区营销模块的详细设计方案，专属于海邻到家项目，未经许可不得外传。